

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 12 | Hashing: Hash Functions, Collisions and Hash Tables

Week 12 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 12 at a glance

Item	Details
Lab title	Hashing: Hash Functions, Collisions and Hash Tables
Week	12 of 16
Prerequisites	Lab 03 (Arrays), Lab 08 (Linked Lists)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab12_Instructor.ipynb , Lab12_Student.ipynb
Next lab	Lab 13: Trees and Binary Trees

1. Learning Objectives

By the end of this lab, you will be able to:

1. Explain how hashing gives **O(1) average** search, insert and delete.
2. Design simple **hash functions** and explain the properties of a good one.
3. Explain **collisions** and resolve them with **separate chaining** and **open addressing (linear probing)**.
4. Explain **load factor** and **rehashing (resizing)**.
5. Implement your own **hash map** from scratch.
6. Use Python's **dict**, **set**, **Counter** and **defaultdict** professionally.
7. Solve classic problems with hashing: frequency counting, two-sum, anagrams and prefix sums.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-1	Explain hash functions and collision resolution	Understand (C2)
CLO-2	Implement a hash table	Apply (C3)
CLO-3	Apply hashing to design efficient solutions	Analyse (C4)

2. Motivation: Finding Anything in One Step

Binary search finds an item among a million sorted items in about 20 steps. Can we do better? **Hashing** can find it in about **one step**, on average, without sorting at all. That is why Python's dictionary and set feel instant even with millions of items.

Analogy: student lockers by roll number

Imagine a university with 1,000 lockers. Instead of searching every locker for Ali's bag, the rule is: **locker number = last 3 digits of the roll number**. Ali (roll 2023-417) always goes straight to locker 417. The rule is the **hash function**; the lockers are the **hash table**. If two students have the same last 3 digits, we have a **collision** and need a plan.

2.1 Hashing in Daily Life

Example	Key → location
Phone contacts / WhatsApp search	Name → contact details

Example	Key → location
Dictionary app	Word → meaning
NADRA / bank systems	CNIC → citizen or account record
Daraz cart	Product ID → quantity
Password check on login	Password → hash stored in the database (never the password itself)
Spell checker	Is this word in the set of valid words?



2.2 Hashing in Industry

- **Databases** use hash indexes and hash joins; **caches** such as Redis and Memcached are giant hash tables.
- **Programming languages:** Python `dict/set`, Java `HashMap`, C++ `unordered_map`, JavaScript objects.
- **Security:** passwords are stored as cryptographic hashes (SHA-256, bcrypt); file downloads are verified with checksums.
- **Git** identifies every commit and file by its SHA hash; **blockchains** link blocks with hashes.
- **Load balancers** use consistent hashing to send each user to the same server.



2.3 Hashing in Research

- **Locality-Sensitive Hashing (LSH)** finds similar images, documents and audio quickly; it powers near-duplicate detection and recommendation systems.
- **Vector search for AI** (RAG, semantic search) uses hashing and graph-based indexes to search millions of embeddings.
- **Bloom filters** (probabilistic hash sets) are used in databases, networking and genomics to save memory.
- **Content-based image retrieval** often uses learned binary hash codes for fast search.



2.4 Why It Matters in 2026–2030



The default tool for fast lookup

In real-world coding interviews, the first improvement to a slow solution is often “**use a hash map**”. As data grows and AI systems search ever-larger collections, hashing (exact and similarity-based) remains one of the most important ideas in computer science.



3. Part A: Hash Functions

A **hash function** converts a **key** (number, string, tuple...) into an **index** of an array called the **hash table** (or bucket array). The table stores the value at that index.

Key → hash → index

key "Ali" --hash function--> 2 --> table[2] = ("Ali", 3.4)

index:	0	1	2	3	4
	[]	[]	[Ali: 3.4]	[]	[]

3.1 A Simple Hash Function for Integers

The **division method**: $h(x) = x \% m$, where m is the table size. With $m = 10$, the values 11, 12, 13, 14, 15 go to indexes 1, 2, 3, 4, 5 (exactly the example in your theory slides). A **prime** table size usually spreads keys better.

```
def h(x, m=10):
    return x % m

for key in [11, 12, 13, 14, 15, 25, 35]:
    print(key, "->", h(key))          # 15, 25, 35 all go to 5:
collisions!
```

3.2 Hashing Strings

```
def bad_hash(s, m):                                # sum of character codes
    return sum(ord(c) for c in s) % m

def poly_hash(s, m, base=31):                      # polynomial rolling hash (used in
    h = 0                                           Java)
    for c in s:
        h = (h * base + ord(c)) % m
    return h

for w in ["listen", "silent", "enlist"]:
    print(f"{w:<7} bad={bad_hash(w, 101):>3} poly={poly_hash(w,
101):>3}")
```

Anagrams have the same characters, so the sum-of-codes hash puts them all in the same bucket. The polynomial hash uses the **position** of each character too, so it spreads them out.

Property of a good hash function	Why
Deterministic	The same key must always give the same index
Fast to compute	Hashing must be $O(1)$ (or $O(\text{length of key})$)
Uniform distribution	Spreads keys evenly to minimise collisions
Uses the whole key	Small differences in keys should give different indexes

Python's built-in hash()

```
print(hash(42), hash("0kara") % 100, hash((1, 2)) % 100)
try:
    hash([1, 2])
except TypeError as e:
    print("Lists cannot be keys:", e)
```

Only **immutable** objects (int, str, tuple of immutables) can be dictionary keys or set items. If a key could change after insertion, its hash would change and it could never be found again.

4. Part B: Collisions and How to Resolve Them

A **collision** happens when two different keys hash to the same index. Collisions are **unavoidable** (there are more possible keys than indexes, the **pigeonhole principle**), so every hash table needs a strategy.

4.1 Strategy 1: Separate Chaining

Each slot holds a **list (chain)** of all key-value pairs that hash there. To search, hash the key and scan only that short chain.

Separate chaining

$m = 5$, keys: 15, 22, 7, 30, 12 ($h = \text{key} \% 5$)

0: [15] -> [30]

1:

2: [22] -> [7] -> [12]

3:

4:

4.2 Strategy 2: Open Addressing (Linear Probing)

All items live in the array itself. If the slot is taken, try the **next** slot ($((i + 1) \% m)$), and keep going until an empty slot is found. Search follows the same path. Deleting requires a special **tombstone** marker so later searches do not stop early.

Linear probing

$m = 7$, insert 10, 17, 24, 5 ($h = \text{key} \% 7$)

10 -> 3 [_, _, _, 10, _, _, _]

17 -> 3 taken, try 4 [_, _, _, 10, 17, _, _]

24 -> 3, 4 taken, try 5 [_, _, _, 10, 17, 24, _]

5 -> 5 taken, try 6 [_, _, _, 10, 17, 24, 5]

	Separate chaining	Linear probing
Where items live	Lists hanging from each slot	Inside the array
Load factor can exceed 1?	Yes	No (must stay below ~0.7)
Deletion	Easy	Needs tombstones
Cache performance	Poorer (pointers)	Excellent (contiguous)
Problem	Long chains if the hash is bad	Clustering: long runs of filled slots

4.3 Load Factor and Rehashing

The **load factor** $\alpha = n / m$ (items $\div$ slots). As α grows, chains get longer (or probe sequences get longer) and operations slow down. When α passes a threshold (e.g., 0.75), the table **doubles its size** and **re-inserts every item** with the new m . Like the dynamic array in Lab 03, this keeps operations **$O(1)$ amortised**.

Operation	Average	Worst (all keys collide)
Search	$O(1)$	$O(n)$
Insert	$O(1)$ amortised	$O(n)$
Delete	$O(1)$	$O(n)$

5. Part C: Build Your Own Hash Map (Separate Chaining)

```
class HashMap:
    def __init__(self, capacity=8):
        self.buckets = [[] for _ in range(capacity)]
        self.size = 0

    def _index(self, key):
        return hash(key) % len(self.buckets)

    def put(self, key, value):
        bucket = self.buckets[self._index(key)]
        for pair in bucket:
            if pair[0] == key:          # key exists: update
                pair[1] = value
                return
        bucket.append([key, value])    # new key
        self.size += 1
        if self.size / len(self.buckets) > 0.75:
            self._resize()

    def get(self, key, default=None):
        for k, v in self.buckets[self._index(key)]:
            if k == key:
                return v
        return default

    def remove(self, key):
        bucket = self.buckets[self._index(key)]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket.pop(i)
                self.size -= 1
                return v
        raise KeyError(key)

    def _resize(self):
        old = self.buckets
        self.buckets = [[] for _ in range(2 * len(old))]
        self.size = 0
        for bucket in old:
```

```

        for k, v in bucket:
            self.put(k, v)                # re-hash into the bigger table

    def __len__(self):
        return self.size

cgpa = HashMap()
for name, g in [("Ali", 3.4), ("Sara", 3.9), ("Hamza", 3.1), ("Zoya", 3.7), ("Bilal",
2.9),
                ("Iqra", 3.5), ("Omar", 3.2)]:
    cgpa.put(name, g)
cgpa.put("Ali", 3.6)                    # update
print(cgpa.get("Ali"), cgpa.get("Nobody", "not found"), len(cgpa), "buckets:",
len(cgpa.buckets))
print("longest chain:", max(len(b) for b in cgpa.buckets))

```



6. Part D: Hashing Tools in Python

```

from collections import Counter, defaultdict

votes = ["Ali", "Sara", "Hamza", "Ali", "Hamza", "Ali"] # class representative
election
tally = Counter(votes)                                # frequency counting in one line
print(tally, tally.most_common(1))

by_city = defaultdict(list)                            # missing keys get a default empty
list
for name, city in [("Ali", "Okara"), ("Sara", "Lahore"), ("Hamza", "Okara")]:
    by_city[city].append(name)
print(dict(by_city))

bscs, bsit = {"Ali", "Sara", "Hamza"}, {"Sara", "Usman"}
print("both:", bscs & bsit, "| only BSCS:", bscs - bsit, "| any:", bscs | bsit)

marks = {"Ali": 81, "Sara": 92}
print(marks.get("Zoya", 0), "Ali" in marks, list(marks.items()))

```

6.1 Cryptographic Hashes (Security)

```

import hashlib
pw = "okara123"
print(hashlib.sha256(pw.encode()).hexdigest())
print(hashlib.sha256("okara124".encode()).hexdigest()) # tiny change -> totally
different

```

Cryptographic hash functions are designed so that you cannot reverse them or find two inputs with the same hash. Websites store this hash (with a random **salt**), not your password. These are different from the fast hash functions used in hash tables.

7. C++ Corner

C++

```
#include <iostream>
#include <unordered_map>
#include <unordered_set>
#include <vector>
#include <list>
using namespace std;

int main() {
    unordered_map<string, double> cgpa;    // hash table (separate
    chaining)
    cgpa["Ali"] = 3.4; cgpa["Sara"] = 3.9;
    cout << cgpa["Sara"] << " " << cgpa.count("Zoya") << endl;    // 3.9 0
    cout << "load factor: " << cgpa.load_factor() << endl;

    unordered_set<int> seen = {1, 2, 3};
    cout << (seen.find(2) != seen.end()) << endl;    // 1

    // A tiny chaining table by hand
    vector<list<int>> table(5);
    for (int k : {15, 22, 7, 30, 12}) table[k % 5].push_back(k);
    for (int i = 0; i < 5; i++) {
        cout << i << ": ";
        for (int k : table[i]) cout << k << " -> ";
        cout << "NULL" << endl;
    }
}
```

8. Common Mistakes

Mistake	Effect	Fix
Using a list as a dictionary key	TypeError: unhashable type	Use a tuple
<code>dict[key]</code> for a missing key	KeyError	Use <code>.get(key, default)</code> or <code>in</code>
Checking <code>x in list</code> inside a loop	Hidden $O(n^2)$	Convert the list to a set first
Assuming dict/set order means sorted	Insertion order, not sorted	Use <code>sorted()</code> when needed
Forgetting to resize a custom hash table	Long chains, $O(n)$ operations	Rehash when load factor > 0.75

Mistake	Effect	Fix
Deleting from a probing table by emptying the slot	Later keys become unreachable	Use a tombstone marker

9. Lab Tasks

Solve all tasks in `Lab12_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Word Frequency Report

Write `top_words(text, k)` that returns the `k` most frequent words (lower-case, punctuation `.,!?` removed) as a list of `(word, count)` tuples, sorted by count descending and then alphabetically. Use a dictionary (do not use `Counter`).

```
top_words("Data is data. Data is power!", 2) → [("data", 3), ("is", 2)]
```

Task 2 Linear Probing Hash Set

Complete `ProbingSet(capacity)` with `add(key)`, `contains(key)` and `remove(key)` using **open addressing with linear probing** and a **DELETED** tombstone. Assume the table never becomes full (no resizing needed).

```
add 10, 17, 24 (capacity 7) → slots 3, 4, 5; remove 17; contains(24) → True
```

Task 3 Group Anagrams

Group words that are anagrams of each other. Use a dictionary whose key is a **canonical form** of the word (its sorted letters as a tuple or string). Return the groups sorted for easy checking: sort each group, then sort the list of groups.

```
["eat", "tea", "tan", "ate", "nat", "bat"] → [
  ["ate", "eat", "tea"], ["bat"],
  ["nat", "tan"]]

```

Task 4 Complete the Hash Map

Extend `HashMap` from Part C into `MyMap` with: `__contains__(key)` (so `key in m` works), `keys()` (list of all keys), `__getitem__`/`__setitem__` (so `m[k]` works; missing keys raise `KeyError`) and `load_factor()`.

Test it with 1,000 keys and check that the load factor stays at or below 0.75.

Task 5 Longest Consecutive Sequence

Given unsorted roll numbers, find the length of the **longest run of consecutive numbers** in $O(n)$ (sorting would be $O(n \log n)$).

Hint: put everything in a set. A number starts a run only if $x - 1$ is not in the set; from each start, count upwards.

```
[100, 4, 200, 1, 3, 2] → 4    (1, 2, 3, 4)
```

Task 6 ● Subarray Sum Equals K (Prefix Sums + Hashing)

A shop's daily profit/loss values are given (can be negative). Count how many **contiguous** periods have a total exactly equal to k .

Hint: keep a running prefix sum and a dictionary counting how many times each prefix sum has occurred. A period ending today has sum k if $\text{prefix} - k$ occurred before. This is $O(n)$ instead of $O(n^2)$.

```
[1, 1, 1], k = 2 → 2
[3, 4, 7, 2, -3, 1, 4, 2], k = 7 → 4
```

Task 7 ● Detect Plagiarism with Rolling Hashes

Two students submitted assignments. Find the **length of the longest common substring** of the two strings using **binary search on the length** and a **rolling (polynomial) hash** set of all substrings of that length (Rabin–Karp idea).

To keep things simple and safe, after finding hash matches, confirm by comparing the actual substrings.

```
longest_common_substring("datastructures", "structural") → 8
("structur")
```



10. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub **DSA-Labs/Lab12** folder.

#	Problem	Level	Concept practised
1	Two Sum	Easy	Hash map of complements
217	Contains Duplicate	Easy	Hash set
242	Valid Anagram	Easy	Frequency counting
387	First Unique Character in a String	Easy	Counting
705	Design HashSet	Easy	Build a hash table
706	Design HashMap	Easy	Chaining (Task 4)

#	Problem	Level	Concept practised
49	Group Anagrams	Medium	Canonical keys (Task 3)
128	Longest Consecutive Sequence	Medium	Set (Task 5)
560	Subarray Sum Equals K	Medium	Prefix sums (Task 6)

11. Home Task

1. Finish all tasks and push `Lab12_Student.ipynb` to GitHub.
2. Add **resizing** to `ProbingSet` (rehash when the load factor exceeds 0.6) and count how many probes are needed on average before and after.
3. Implement **quadratic probing** ($i, i+1^2, i+2^2, \dots$) and **double hashing**, and compare clustering with linear probing on 5,000 random keys.
4. **Mini-project: Student Record System.** Store at least 1,000 generated student records in your own `MyMap`, keyed by roll number. Support add, search, update, delete, and a report of the longest chain and load factor. Compare search time with a plain list.
5. Solve at least 3 LeetCode problems from the list.

12. Viva Questions

1. What is hashing? Why is lookup $O(1)$ on average?
2. What is a hash function? List three properties of a good one.
3. What is a collision? Why are collisions unavoidable?
4. Explain separate chaining with an example.
5. Explain linear probing. What is clustering?
6. Why does deletion in open addressing need a tombstone?
7. What is the load factor? What happens when it gets too high?
8. Why can a Python list not be a dictionary key?
9. What is the worst-case complexity of a hash table lookup and when does it happen?
10. How is a cryptographic hash different from a hash-table hash?



13. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain



Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.



14. References and Further Learning

- Python **collections** (Counter, defaultdict): docs.python.org/3/library/collections.html
- Python **hashlib**: docs.python.org/3/library/hashlib.html
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).

- Visualise code step by step: **pythontutor.com**; animated data structures: **visualgo.net**.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.